

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto Monagua

87 / 100 PUNTUACION GLOBAL	ESTADO: ACEPTABLE CON DETALLES POR CORREGIR Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	--

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 128	
Plantillas que heredan ({% extends %}): 84	
Plantillas con fragmentos reusables ({% include %}): 17	
Bloques definidos ({% block %}): 86	
Uso de static ({% load static %}): 75	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 9	
Rutas/endpoints totales definidos en el servidor: 100	
- .\autenticacion\urls.py	
- .\catalogo\urls.py	
- .\comunidad\urls.py	
- .\core\urls.py	

- - .\notificaciones\urls.py

- - .\pagos\urls.py

- - .\promociones\urls.py

- - .\reservas\urls.py

- - .\usuarios\urls.py

3. 3. Consumo de API REST, carga y errores

100 / 100

Excelente. Se consumen APIs y se manejan correctamente los errores de conexión y estados de carga.

- - .\static\js\auth\países_ciudades.js (Frontend JS): 2 llamadas (Con manejo de errores y estado de carga)

4. 4. Estilos y metodologías CSS (BEM / Atómico)

70 / 100

Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.

- Archivos CSS en la carpeta estática: 5
- Selectores con estructura BEM (___ o --) detectados: 0
- Estructuras de diseño atómico (clases utilitarias Bootstrap): 4374
- Estilos en línea (style='...') en HTML: 725 (se sugiere minimizarlos)

5. 5. Binding de datos reactivo y DOM

100 / 100

Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.

- Archivos JavaScript escaneados: 15
- Escrituras dinámicas en el DOM (binding de salida): 23
- Escuchadores de eventos de entrada (binding de entrada): 5

6. 6. Validación de formularios

100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 5

• Llamadas a validación segura en vistas (.is_valid()): 17	
• Atributos de validación HTML5 en plantillas frontend: 101	
• - .\autenticacion\forms.py	
• - .\catalogo\forms.py	
• - .\comunidad\forms.py	
• - .\reservas\forms.py	
• - .\usuarios\forms.py	
7. 7. Configuración limpia por Variables de Entorno	10 / 100
<i>Datos sensibles expuestos críticamente en el repositorio. Falta el uso de variables de entorno.</i>	
• Archivo .env o .env.example presente: No	
• Llamadas a variables de entorno en código (os.getenv/decouple): 6	
• [ALERTA] Credencial expuesta: SECRET_KEY expuesta en settings.py	
• [ALERTA] Credencial expuesta: EMAIL_HOST_PASSWORD expuesta en settings.py	
8. 8. Jerarquía y organización del código fuente	100 / 100
<i>Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.</i>	
• - vistas (views.py): 11	
• - modelos (models.py): 10	
• - formularios (forms.py): 5	
• - rutas (urls.py): 9	
• - plantillas (.html): 128	
• - recursos estaticos (.css/.js): 20	
• - tests (tests.py): 11	
9. 9. Pruebas unitarias sobre componentes	100 / 100

¡Excelente cobertura de pruebas! Se encontraron 76 métodos de prueba estructurados.

- Archivos de prueba detectados: 11

- Clases de Test definidos: 13

- Funciones de prueba (test_*): 76

- - .\autenticacion\tests.py

- - .\catalogo\tests.py

- - .\comunidad\tests.py

- - .\core\tests.py

- - .\guias\tests.py

- - .\IA\tests.py

- - .\notificaciones\tests.py

- - .\pagos\tests.py

- - .\promociones\tests.py

- - .\reservas\tests.py

- - .\usuarios\tests.py

10. 10. Optimización de carga (Lazy loading / split)

75 / 100

Optimización parcial. Añade loading='lazy' en imágenes pesadas de tus catálogos o galerías.

- Imágenes o recursos con carga diferida (loading='lazy'): 4

- Uso de scripts asíncronos o diferidos (async/defer): 0

11. 11. Documentación (JSDoc / Comentarios)

100 / 100

Código fuente ampliamente documentado con estándares JSDoc y Docstrings descriptivos.

- Archivos de código escaneados: 92 Python, 15 JavaScript

- Docstrings de documentación en Python: 104

- Comentarios estilo JSDoc en JavaScript: 4

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 117
- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 1263

RECOMENDACIONES CLAVE PARA MEJORA:

- Migrar las variables SECRET_KEY y EMAIL_HOST_PASSWORD a variables de entorno (.env).
- Reducir el uso de estilos en linea (style="...") en las plantillas HTML (trasladar a CSS).
- Añadir loading="lazy" a imagenes y async/defer a scripts.
- Mejorar documentacion de metodos complejos del frontend usando estandar JSDoc.